

Jak działa Docker?

System operacyjny to oprogramowanie zarządzające zasobami sprzętowymi komputera. Gdy Docker uruchamiany jest na systemach z rodziny Linux, tworzy subprocess w ramach systemu operacyjnego, działający we własnej przestrzeni nazw (namespace) i katalogów.

Co to oznacza?

Powłoka systemowa umożliwia komunikację z katalogami, plikami oraz uruchamianie procesów. Wewnątrz kontenera Docker tworzy izolowane środowisko emulujące powłokę systemową. Zamiast operować na ścieżkach głównego systemu operacyjnego, korzysta ze ścieżek i przestrzeni nazw udostępnianych przez silnik Dockera.

Subprocess wywoływany przez Docker różni się od wątku w aplikacji wielowątkowej. Wątek, gdy nie wykonuje żadnego zadania, pozostaje zatrzymany w pętli i przechowuje w pamięci podręcznej dane zapisane w momencie ostatniego wykonywania. Subprocess natomiast może działać nieprzerwanie bez obciążania pamięci podręcznej i sam zarządza przydzielonymi zasobami, dostosowując je do swoich potrzeb.

Maszyna wirtualna również działa nieprzerwanie i samodzielnie zarządza zasobami sprzętowymi. Różnica polega na tym, że przy jej uruchamianiu musimy z wyprzedzeniem przydzielić zasoby (CPU, RAM, dysk) dla wirtualizowanego systemu. Wynika to m.in. z tego, że system operacyjny zapisze plik tylko wtedy, gdy dysponuje wymaganą przestrzenią dyskową — w przeciwnym razie zwróci błąd.

Docker działa inaczej tj. sub-procesem uruchomionym bezpośrednio w systemie operacyjnym hosta, ale tworzy własną przestrzeń nazw. Polecenia wykonywane wewnątrz kontenera są przechwytywane i realizowane przez system operacyjny hosta, w wydzielonej przestrzeni Dockera. To właśnie stanowi istotę konteneryzacji.

Docker Desktop to aplikacja z graficznym interfejsem użytkownika, ułatwiająca zarządzanie Dockerem.

Różnica między obrazem, kontenerem a Dockerem

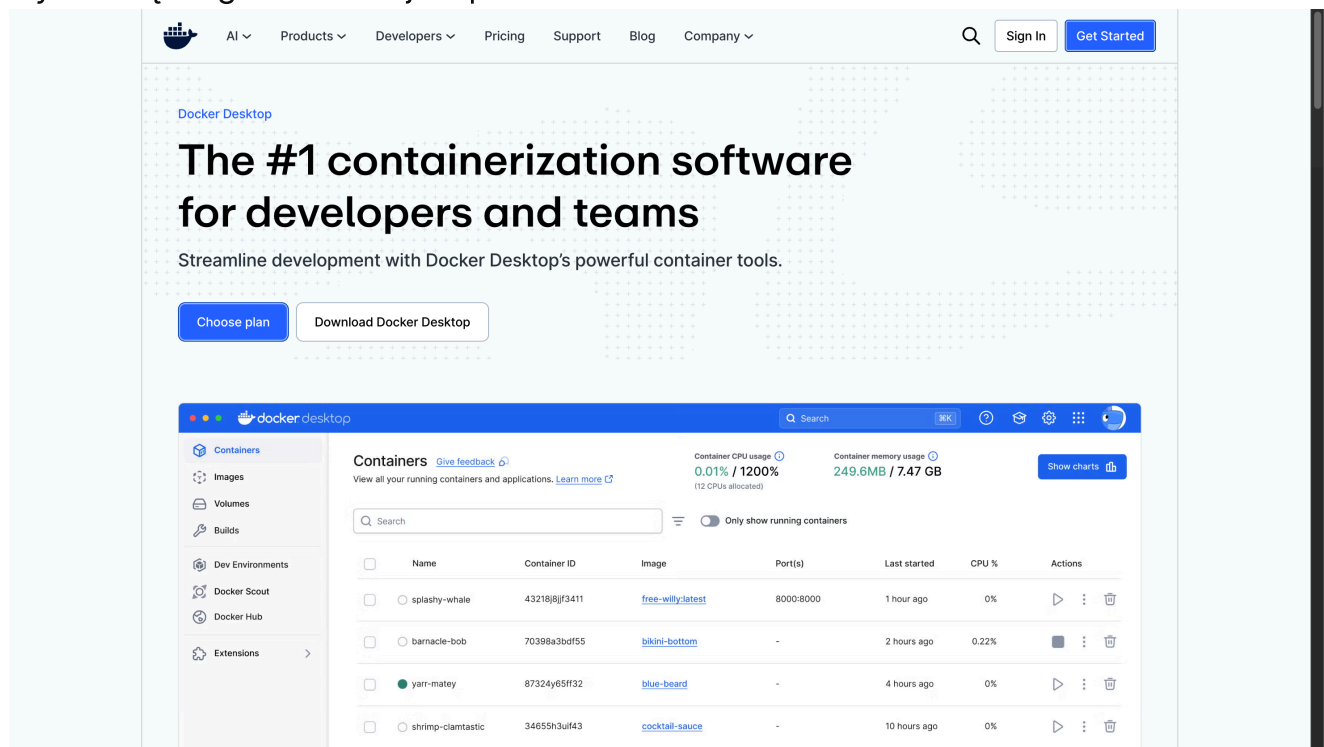
Kontener to izolowana przestrzeń nazw z własną listą zmiennych środowiskowych. Kontener ma własną sieć, lecz nie jest ona jego integralną częścią (inaczej niż w głównym systemie operacyjnym). Dlatego po usunięciu kontenera przypisana do niego sieć może pozostać nieusunięta. Stąd panuje przekonanie, że Docker zostawia „śmieci”, którymi warto nauczyć się zarządzać. Docker Desktop częściowo w tym pomaga.

Obraz (image) to szablon zawierający aplikację wraz z jej konfiguracją – budowany na bazie obrazów dostępnych w repozytorium [Docker Hub](https://hub.docker.com/). Uruchomiony obraz staje się kontenerem tj. działającą instancją aplikacji.

Docker Compose pozwala uruchamiać wiele kontenerów jednocześnie: jeśli plik konfiguracyjny zawiera dwie zależności, powstaną dwa osobne kontenery. Dlatego warto rozważyć stosowanie bardziej granularnych obrazów, aby łatwiej zarządzać poszczególnymi komponentami.

Pobieranie i instalacja Docker Desktop na Windows

Docker Desktop można pobrać ze strony docker.com, wybierając wersję odpowiednią dla naszego systemu operacyjnego. Warto zaznaczyć, że Docker Desktop nie wymaga zakładania konta – do podstawowych operacji, które na początek w zupełności wystarczą. Logowanie nie jest potrzebne.



Składniki Docker Desktop

- **Docker Engine** – rdzeń odpowiedzialny za zarządzanie kontenerami
- **Docker CLI** – interfejs wiersza poleceń do obsługi Dockera
- **Docker Compose** – narzędzie do definiowania i uruchamiania aplikacji wielokontenerowych
- **Docker Desktop UI** – graficzny interfejs użytkownika

Docker Compose

Wyobraźmy sobie kompozytora, który za pomocą nut tworzy partyturę. Dopiero gdy orkiestra zaczyna grać, jesteśmy w stanie usłyszeć muzykę. Podobnie działa Docker

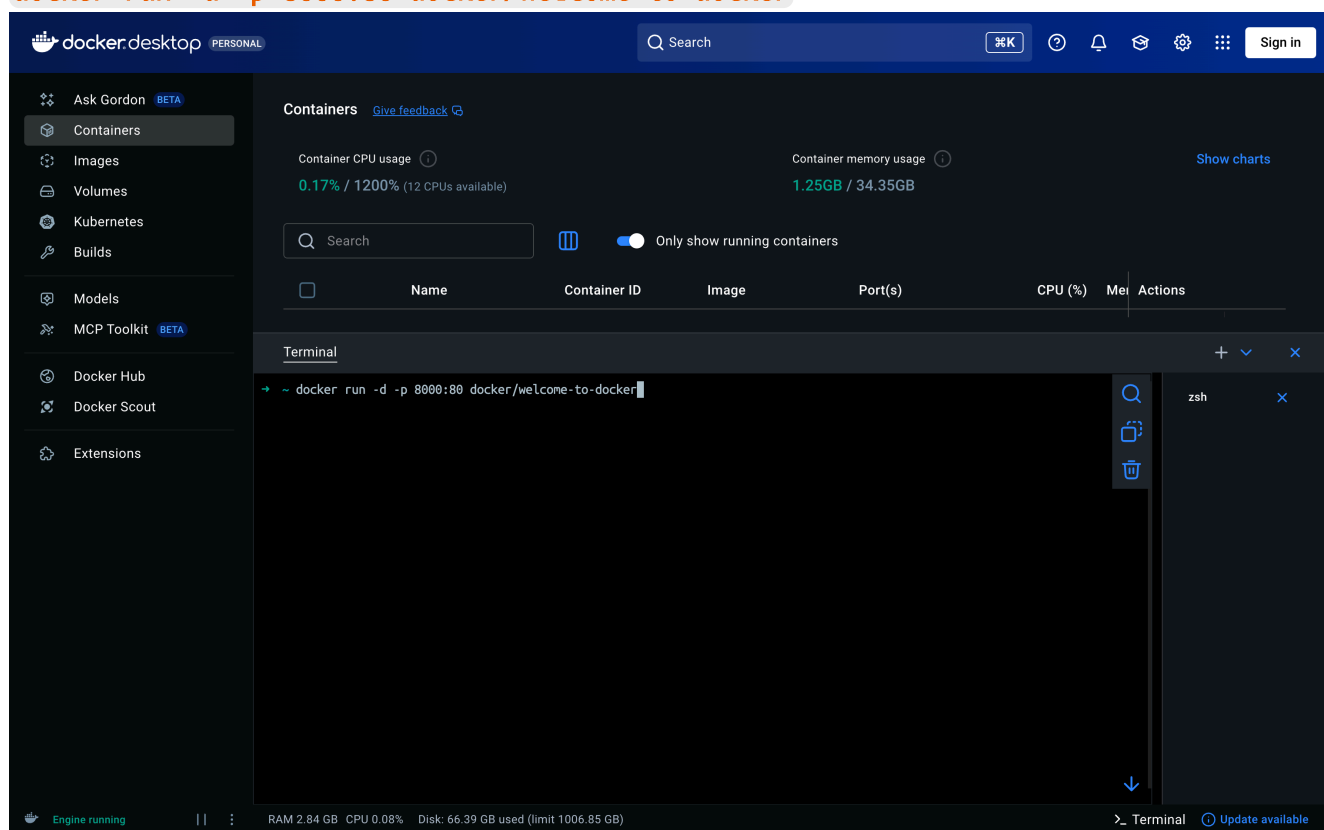
Compose tj. na podstawie konfiguracji zawartej w pliku `.yaml` uruchamia kontenery w Dockerze, jednocześnie wdrażając konfigurację sieci, wolumenów i innych zależności.

Korzystanie z Docker Compose nie jest obowiązkowe. Osobiście za jego pomocą uruchamiam jedynie własne aplikacje, natomiast rozwiązania open-source staram się uruchamiać i konfigurować bezpośrednio w Docker Engine – za pomocą CLI (Command Line Interface) lub Docker Desktop.

Jak uruchomić kontener?

Kontener uruchamiamy w terminalu, wpisując:

```
docker run -d -p 8000:80 docker/welcome-to-docker
```



Gdzie:

- `-d` – uruchamia kontener w tle (bez strumieniowania wyjścia na terminal),
 - `-p` – mapuje port wewnętrzny kontenera na port widoczny z poziomu systemu operacyjnego hosta,
 - `docker/welcome-to-docker` – ścieżka do obrazu na Docker Hub, który zostanie pobrany i uruchomiony.
-

Czym jest WSL (Windows Subsystem for Linux)?

WSL (Windows Subsystem for Linux) umożliwia uruchamianie środowiska Linux bezpośrednio w systemie Windows tj. bez konieczności instalowania tradycyjnej maszyny wirtualnej.

- **WSL 1:** warstwa kompatybilności tłumacząca wywołania systemowe Linuksa na odpowiadające im funkcje Windows.
- **WSL 2:** wykorzystuje lekką maszynę wirtualną z pełnym jądrem Linux, co zapewnia lepszą wydajność i pełną kompatybilność.

Jak zainstalować WSL na Windows?

```
wsl --install
```

Uruchomienie AI na lokalnej maszynie

Tworzenie wolumenu przechowującego pobrane modele LLM:

```
mkdir ollama_images
```

Uruchomienie kontenera z obrazem Ollama:

```
docker run -d -v ${PWD}/ollama_images:/root/.ollama -p 11434:11434 --name ollama_ai ollama/ollama
```

Sprawdzenie logów:

```
docker logs -f ollama_ai
```

W razie potrzeby można dodać przełączniki alokujące GPU.

Uruchomienie zatrzymanego kontenera:

```
docker start ollama_ai
```

Uruchomienie interaktywnej sesji z modelem DeepSeek wewnątrz kontenera:

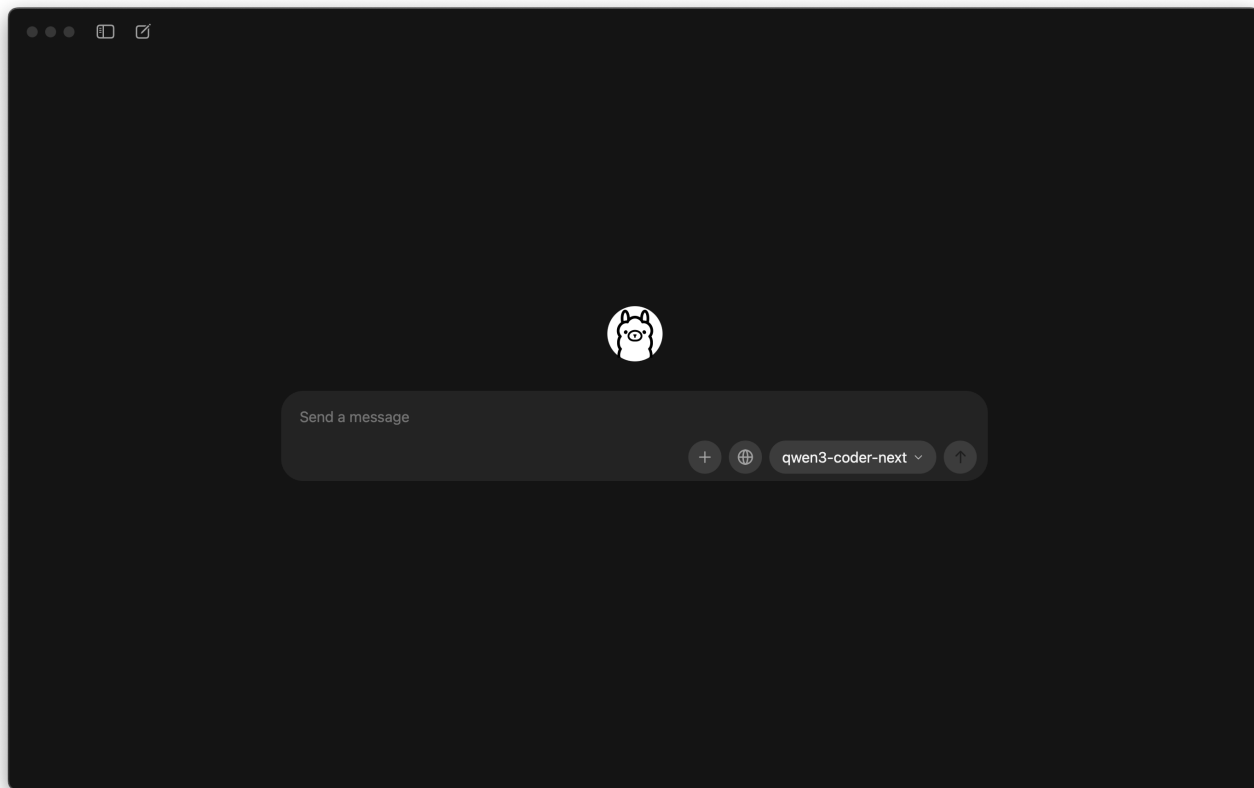
```
docker exec -it ollama_ai ollama run deepseek-r1:1.5b
```

Powyższe polecenie otwiera interaktywną sesję z modelem AI w kontenerze, tunelując wejście i wyjście na terminal hosta.

Model DeepSeek-r1 to model typu *instruct* z dodatkową zdolnością *reasoning* — można mu wydawać polecenia (instrukcje), którymi kieruje się przy udzielaniu odpowiedzi. Modele, które nie zostały douczone w trybie *instruct*, jedynie dopełniają tekst na podstawie prawdopodobieństwa wystąpienia kolejnych tokenów.

Alternatywa: Ollama bez Dockera

Ollama można zainstalować również bezpośrednio w systemie operacyjnym — bez konieczności korzystania z Dockera. To prostsze rozwiązanie, jeśli chcemy jedynie eksperymentować z modelami AI na lokalnej maszynie.



macOS:

```
brew install ollama
```

Alternatywnie można pobrać instalator ze strony ollama.com.

Linux:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Windows:

Pobieramy instalator ze strony ollama.com i uruchamiamy go — proces instalacji jest analogiczny do instalacji innych aplikacji w systemie Windows.

Uruchomienie modelu:

Po zainstalowaniu Ollamy uruchamiamy model bezpośrednio z terminala:

```
ollama run deepseek-r1:1.5b
```

Przy pierwszym uruchomieniu model zostanie automatycznie pobrany. Każde kolejne uruchomienie korzysta już z lokalnie zapisanego modelu.

W wersji natywnej (bez Dockera) modele przechowywane są domyślnie w katalogu `~/.ollama/models`. W wersji kontenerowej lokalizacja zależy od zamontowanego wolumenu.

Podsumowanie

W ramach tego poradnika omówiliśmy kluczowe zagadnienia związane z Dockerem i konteneryzacją:

1. **Wyjaśniliśmy zasadę działania Dockera** – jako sub-procesu systemu operacyjnego tworzącego izolowane przestrzenie nazw (kontenery), w odróżnieniu od maszyn wirtualnych wymagających z góry przydzielonych zasobów.
2. **Omówiliśmy różnicę między obrazem, kontenerem a Docker Compose** – obraz to szablon, kontener to jego uruchomiona instancja, a Docker Compose pozwala orkiestrować wiele kontenerów jednocześnie.
3. **Zainstalowaliśmy Docker Desktop** – aplikację z graficznym interfejsem ułatwiającą zarządzanie kontenerami, dostępną na Windows, macOS i Linuksie.
4. **Uruchomiliśmy pierwszy kontener** – demonstrując podstawowe flagi (`-d`, `-p`) oraz mapowanie portów.
5. **Uruchomiliśmy lokalny model AI (Ollama)** – zarówno w kontenerze Docker, jak i natywnie w systemie operacyjnym, pokazując praktyczne zastosowanie konteneryzacji.

Docker stanowi fundament do uruchamiania wielu narzędzi omawianych na warsztatach – w tym platformy n8n w wersji self-hosted.